

Assignment - 1 (Lab)

1, 2, 3 :- Construct the binary tree and perform ~~in~~ order, pre-order, post order

Class node

int data;

Node left, right;

Node (int value) {

data = value;

left = right = null; }

public class BinaryTreeTraversal {

Static Node Create Tree (int [] arr, int start, int end) {

if (start > end)

return null;

int mid = (start + end) / 2;

Node root = new

Node (arr[mid]);

root.left = create Tree (arr, start, mid-1);

root.right = create Tree (arr, mid+1, end);

return root; }

Static void preOrder (Node root) {

if (root == null)

return;

System.out.print (root.data + " ");

preOrder (root.left);

preOrder (root.right); }

```
System.out.print (root.data + " ");  
public static void main main (String[] args) {  
    int[] T = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};  
    Node root = createTree (T, 0, T.length - 1);  
    System.out.print ("Pre-order");  
    preOrder (root);  
    System.out.print ("Post-order");  
    postOrder (root); } }
```

Output

Pre-order: 6 4 2 1 3 5 8 7 9 10

Post-order: 1 3 2 5 4 7 10 9 8 6

Pseudocode

Algo Binary Tree Traversal

structure Node

data

left

right

END structure

function create tree (arr, start, end)

if start > end then

return null

end if

mid $\leftarrow$ (start + end) / 2

node $\leftarrow$ New Node (arr[mid])

node.left $\leftarrow$ create-tree (arr, start, mid - 1)


```
node.right ← create-tree (arr, mid + 1, end)
return node
end function

procedure pre_order (root)
  if root = null then
    return
  end if
  print root.data
  pre_order (root.left)
  pre_order (root.right)
end procedure

procedure post_order (root)
  if root = null then
    return
  end if
  post_order (root.left)
  post_order (root.right)
  print root.data
end procedure

begin
  arr ← [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
  root ← create-tree (arr, 0, length (arr) - 1)
  print "Pre-order: "
  pre_order (root)
  print "Post-order: "
  post_order (root)
end
```

4.5.6 : Perform in-order, level order, find height of binary tree of n -th inputs.

Pseudocode

procedure inorder (root)

 if root = null then

 return

 end if

 inorder (root.left)

 print root.data

 inorder (root.right)

end procedure

procedure levelorder (root)

 if root = null then

 return

 end if

 create empty queue Q

 Enqueue (Q, root)

 while Q is not empty DO

 temp ← Dequeue (Q)

 print temp.data

 if temp.left ≠ Null then

 enqueue (Q, temp.left)

 end if

 if temp.right ≠ null then

 enqueue (Q, temp.right)

 end if

 end while

end procedure.


```
function height (root)
    if root = null then
        return 0
    end if
    left Height ← Height (root.left)
    right Height ← Height (root.right)
    return 1 + max (left Height, right Height)
end function
```

Code :

```
import java.util.*;
```

```
class Node {
```

```
    int data;
```

```
    Node left, right;
```

```
    Node (int value) {
```

```
        data = value;
```

```
        left = right = null; } }
```

```
public class Binary Tree All operations {
```

```
    static Node create Tree (int [] arr, int start, int end) {
```

```
        if (start > end)
```

```
            return null;
```

```
            int mid = (start + end) / 2;
```

```
            Node root = new
```

```
Node (arr [mid]);
```

```
root.left = create Tree (arr, start, mid-1);
```

```
root.right = create Tree (arr, mid+1, end);
```

```
return root; }
```

```
Static void Preorder (Node root) {  
    if (root == null)  
        return;  
    System.out.print (root.data + " ");  
    preorder (root.left);  
    preorder (root.right); }  
}
```

```
Static void inorder (Node root) {  
    if (root == null) .  
        return;  
    inorder (root.left);  
    System.out.print (root.data + " ");  
    inorder (root.right); }  
}
```

```
Static void post order (node root) {  
    if (root == null)  
        return;  
    postorder (root.left);  
    postorder (root.right);  
    System.out.print (root.data + " "); }  
}
```

```
Static int height (Node root) {  
    if (root == null) return 0;  
    return 1 +
```

```
    Math.max (height (root.left), height (root.right)); }  
}
```

```
Static void printCurrentLevels (Node root, int level) {  
    if (root == null) return;  
    if (level == 1)
```



```
System.out.print(nroot.data + " ");
else {
    printCurrentLevel(nroot.left, level - 1);
    printCurrentLevel(nroot.right, level - 1);
}
static void levelOrder(Node nroot) {
    int n = height = height(nroot);
    for (int i = 1; i <= n; i++) {
        printCurrentLevel(nroot, i);
    }
}
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter no of elements:");
    int n = sc.nextInt();
    int[] arr = new int[n];
    System.out.println("Enter elements in sorted order:");
    for (int i = 0; i < n; i++) {
        arr[i] = sc.nextInt();
    }
    Node nroot = createTree(arr, 0, n - 1);
    System.out.print("Preorder Traversal:");
    preOrder(nroot);
    System.out.print("\nInorder Traversal:");
    inOrder(nroot);
    System.out.print("\nPostorder Traversal:");
    postOrder(nroot);
    System.out.print("\nLevel order Traversal:");
    levelOrder(nroot);
    System.out.println("\nHeight of Tree " + height(nroot));
    sc.close();
}
```

Sample output :

Enter no. of nodes : 5

Enter elements : 1 2 3 4 5

output

Pre order Traversal : 1 2 4 5 3

Height of Tree : 3